



VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | **SAFU** |
CEX Listing | Marketing

MADE IN CANADA

NOAH DAO

AUDIT

SECURITY ASSESSMENT

28TH SEPTEMBER 2026

For



Making Blockchain, Defi And Web3 A Safer Place.



**Smart
Check**



SLITHER



**TRAIL
OF BITS**

MythX



@VB_Audit



@Vitalblock



@VB-Audit



www.vitalblock.org

Table Of Content

Chapter 1 Introduction

About the Audit Target	1
1. <u>Disclaimer</u>	1
2. <u>Procedure of Auditing</u>	2
1. <u>Security Issues</u>	2
2. <u>Additional Recommendation</u>	3
3. <u>Security Model</u>	3

Chapter 2 Findings

<u>Security Issue</u>	4
1. <u>Potential fee circumvention due to malicious delegations</u>	4
1. <u>Recommendation</u>	7
1. <u>Remove the redundant function receive()</u>	7
2. <u>Add checks for the value s in the function _recover()</u>	7
2. <u>Note</u>	9
1. <u>Potential centralization risks</u>	9

INTRODUCTION

Auditing Firm	 VITAL BLOCK SECURITY
Client Firm	 NOAH DAO
Methodology	Automated Analysis, Manual Code Review
Language	Solidity
Contract Code	Noahstaking.SOL Yieldfarming.SOL
Source Code Light	Public Source
Centralization	Active ownership
Blockchain	 ARC CHAIN
Contract Name	NOAH DAO
Compiler	<code>pragma solidity ^0.8.20;</code>
Website	www.noahdao.xyz.
Telegram	@zipcateth
Twitter / X	@zipcatEth
Prelim Report Date	September 28 th 2026
Final Report Date	September 28 TH 2026

 Verify the authenticity of this report on our GitHub Repo: <https://www.github.com/vital-block>

Document Properties

Client	NOAH DAO
Title	Smart Contract Audit Report
Target	NOAH DAO
Audit Version	1.0
Author	Akhmetshin Marat
Auditors	Akhmetshin Marat, James BK, Benny Matin
Reviewed by	Dima Meru
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	SEPTEMBER 28 TH , 2026	James BK	Final Released
1.0-AP	SEPTEMBER 28 TH , 2026	Jimmy Cole	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Akhmetshin Marat
Phone 	+44 7944 248057
Email	info@vitalblock.org



Digitally signed by: Vital Block Security (www.vitalblock.org)

SHA1 digest of public Audit: CHTYUU00908BC6DE0963CE05E890RYHTDR4537

Date: 2026-09-26 03:00:05+0000

In the following, we show the specific pull request and the commit hash value used in this audit.

- **NOAH DAO** (NEO099750)

About Vital Block Security

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We can audit custom smart contracts, DApps, Rust, NFTs, etc (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/vitalblock>),

Twitter / X: (http://twitter.com/Vb_Audit), or Email (info@vitalblock.org).

Table 1.2: Vulnerability Severity Classification

Impact	High			
	Medium	Critical	High	Medium
	Low	High	Medium	Low
		Medium	Low	Low
		High	Medium	Low
		Likelihood		

Methodology (1)

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

SCOPE OF WORK

Vital Block Security will conduct the smart contract audit of its **NOAH DAO** source code. The audit scope of work is strictly limited to mentioned .SOL file only.

```
O.Noahstaking.sol  
O.Noahyieldfarm.sol
```

 **External contracts and/or interfaces dependencies are not checked due to being out of scope.**

Verify audited contract code Repo.

PUBLIC CONTRACT CODE

```
O.Noahstaking.sol  
O.Noahyieldfarm.sol
```

AUDIT METHODOLOGY

Smart contract audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block Security auditing process and methodology:

CONNECT

- The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.

AUDIT

- Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:
 - Remix IDE Developer Tool
 - Open Zeppelin Code Analyzer
 - SWC Vulnerabilities Registry
 - DEX Dependencies, e.g., Pancakeswap, Uniswap
- Simulations are performed to identify centralized exploits causing contract and/or trade locks.
- A manual line-by-line analysis is performed to identify contract issues and centralized privileges. We may inspect below mentioned common contract vulnerabilities, and centralized exploits:

Centralized Exploits	○ Token Supply Manipulation ○ Access Control and Authorization ○ Assets Manipulation ○ Ownership Control ○ Liquidity Access ○ Stop and Pause Trading ○ Ownable Library Verification
----------------------	---

Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

 It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.

Table 1.0 The Full Audit Checklist

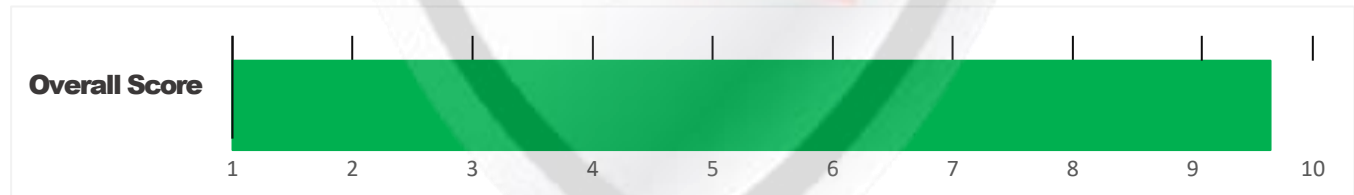
Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

EXECUTIVE SUMMARY

Vital Block Security has performed the automated and manual analysis of the **NOAH DAO .Sol** code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical ! 	Major " 	Medium # 	Minor \$ 	Unknown % 
Open	0	0	0	0	1
Acknowledged	0	0	0	0	3
Resolved	2	0	1	2	0
Noteworthy OnlyOwner Privileges	Set Taxes and Ratios, Airdrop, Set Protection Settings, Set Reward Properties, Set Reflector Settings, Set Swap Settings, Set Pair and Router				

CRYSTAL STONE Smart contract has achieved the following score: **98.5 %**



-  Please note that smart contracts deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.
-  Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization is elevated.

There are some well-intended reasons have privileged roles, such as:

- **Privileged roles can be granted the power to `pause()` the contract in case of an external attack.**
- **Privileged roles can use functions like, `include()`, and `exclude()` to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to list on an exchange.**

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization-related losses are increasing in frequency and magnitude.

- **The client can lower centralization-related risks by implementing below mentioned practices:**
- **Privileged role's private key must be carefully secured to avoid any potential hack.**
- **Privileged role should be shared by multi-signature (multi-sig) wallets.**
- **Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.**
- **Renouncing the contract ownership, and privileged roles.**
- **Remove functions with elevated centralization risk.**

 **Understand the project's initial asset distribution. Assets in the liquidity pair should be locked.**

Assets outside the liquidity pair should be locked with a release schedule.

RISK CATEGORIES

Smart contracts are generally designed to hold, approve, and transfer tokens. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

Risk Type	Definition
Critical ! 	These risks could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
Major " 	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium # 	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re-entrancy-related vulnerabilities should be fixed to deter exploits.
Minor \$ 	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown % 	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.

OPTIMIZATIONS | NOAH DAO

ID	Title	Category	Status
NHP	Logarithm Refinement Optimization	Gas Optimization	Acknowledged
NMJ	Checks Can Be Performed Earlier	Gas Optimization	Acknowledged
MDE	Unnecessary Use Of SafeMath	Gas Optimization	Acknowledged
NWQ	Struct Optimization	Gas Optimization	Acknowledged
NBR	Unused State Variable	Gas Optimization	Acknowledged

Executive Summary

A comprehensive security review was conducted on the **Noahdao Staking Contract** codebase. The audit focused on smart contract safety, access control logic, economic attack vectors, ERC-20 token standard compliance, arithmetic precision/overflow safety, reentrancy vulnerabilities, and gas efficiency.

Key Findings

Overall, these contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table [2.1](#)), 1 High-severity, 2 medium-severity vulnerabilities, 1 low-severity vulnerabilities, and 1 informational recommendations.

Table 2.1: Key NOAH DAO Audit Findings

File / Modules Analyzed	Function / Purpose	Risk Classification
Staking.sol	Precision Loss / Integer Division in Reward Rate Calculation	Mathematical & Economic Flaw
Staking.sol	Centralization Risk & Unrestricted Owner Withdrawals (emergencyWithdraw)	Access Control & Rug-Pull Risk
Staking.sol	Monitoring & Off-Chain Indexing	Missing Event Emissions
Staking.sol	Gas Optimization	Storage Layout Packing
Noah.sol	Asset Loss Risk	Missing Token Recovery Mechanism

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to page [10](#) for details.

AUTOMATED ANALYSIS

Symbol	Definition
	Function modifies state
	Function is payable
	Function is internal
	Function is private
	Function is important

```

** NOAH DAO ** | Interface | | |
| L | totalSupply | External ! | ! | NO ! |
| L | decimals | External ! | ! | NO ! |
| L | symbol | External ! | ! | NO ! |
| L | name | External ! | ! | NO ! |
| L | getOwner | External ! | | NO ! |
| L | balanceOf | External ! | ! | NO ! |
| L | transfer | External ! | " ! ! | NO ! |
| L | allowance | External ! | ! | NO ! |
| L | approve | External ! | " ! ! | NO ! |
| L | transferFrom | External ! | " | NO ! |
| | | | |
**IFactoryV2** | Interface | | |
| L | getPair | External ! | | NO ! |
| L | createPair | External ! | " | NO ! |
| | | | |
**IV2Pair** | Interface | | |
| L | factory | External ! | | NO ! |
| L | getReserves | External ! | | NO ! |
| L | sync | External ! | " | NO ! |

```

|||||

| ****IRouter01**** | Interface | |||

| L | factory | External ! | |NO!|

| L | ETH | External ! | |NO!|

| L | addLiquidityETH | External ! | # |NO!|

| L | addLiquidity | External ! | " |NO!|

| L | swapExactETHForTokens | External ! | # |NO!|

| L | getAmountsOut | External ! | |NO!|

| L | getAmountsIn | External ! | |NO!|

|||||

| ****IRouter02**** | Interface | IRouter01 |||

| L | swapExactTokensForETHSupportingFeeOnTransferTokens | External ! | " |NO!|

| L | swapExactETHForTokensSupportingFeeOnTransferTokens | External ! | # |NO!|

| L | swapExactTokensForTokensSupportingFeeOnTransferTokens | External ! | " ! |NO!|

| L | swapExactTokensForTokens | External ! | " |NO!|

|||||

| ****Protections**** | Interface | |||

| L | checkUser | External ! | " ! |NO!|

| L | setLaunch | External ! | " ! |NO!|

| L | setLpPair | External ! | " ! |NO!|

| L | **NOAH DAO** | External ! | " |NO!|

| L | removeSniper | External ! | " |NO!|

|||||

| ****Cashier**** | Interface | |||

| L | setRewardsProperties | External ! | " |NO!|

| L | tally | External ! | " |NO!|

| L | load | External ! | # |NO!|

| L | cashout | External ! | " |NO!|

| L | giveMeWelfarePlease | External ! | " |NO!|

| L | getTotalDistributed | External ! | |NO!|

| L | getUserInfo | External ! | |NO!|

| L | getUserRealizedRewards | External ! | |NO!|

```

| L | getPendingRewards | External ! | | | NO ! |
| L | initialize | External ! | " | NO ! |
| L | getCurrentReward | External ! | | | NO ! |
|||||
| **ETH** | Implementation | SafeMath |||
| L | <Constructor> | Public ! | # | NO ! |
| L | transferOwner | External ! | " | onlyOwner |
| L | renounceOwnership | External ! | " | NO ! |
| L | setOperator | Public ! | " | NO ! |
| L | renounceOriginalDeployer | External ! | " | NO ! |
| L | <Receive Ether> | External ! | # | NO ! |
| L | totalSupply | External ! | | | NO ! |
| L | decimals | External ! | | | NO ! |
| L | symbol | External ! | | | NO ! |
| L | name | External ! | | | NO ! |
| L | getOwner | External ! | ! | NO ! |
| L | balanceOf | Public ! | ! | NO ! |
| L | allowance | External ! | ! | NO ! |
| L | approve | External ! | " ! | NO ! |
| L | _approve | Internal $ | " | |
| L | approveContractContingency | Public ! | " ! | onlyOwner |
| L | transfer | External ! | " | NO ! |
| L | transferFrom | External ! | " | NO ! |
| L | setNewRouter | External ! | " | onlyOwner |
| L | setLpPair | External ! | " | onlyOwner |
| L | setInitializers | External ! | " | onlyOwner |
| L | isExcludedFromFees | External ! | | | NO ! |
| L | isExcludedFromDividends | External ! | | | NO ! |
| L | isExcludedFromProtection | External ! | | | NO ! |
| L | setDividendExcluded | Public ! | " | onlyOwner |
| L | setExcludedFromFees | Public ! | " | onlyOwner |

```

NK-01 Key Findings

Category	Severity	Target	Status
Mathematical & Economic Flaw	MEDIUM	Administrative Fee Setters	Acknowledged

Precision Loss / Integer Division in Reward Rate Calculation

Detailed Description

When computing `rewardPerToken()` or distribution over short blocks/durations, division performed prior to multiplication (or small scale multipliers) causes truncation errors down to zero. Small stakers or frequent withdrawers lose precision rewards due to integer division truncation in Solidity

```
// FIXING REMARK CODE EXAMPLE
function rewardPerToken() public view returns (uint256) {
    if (_totalSupply == 0) {
        return rewardPerTokenStored;
    }
    return rewardPerTokenStored + (
        ((lastTimeRewardApplicable() - lastUpdateTime) * rewardRate * 1e18) / _totalSupply
    );
}
```

Impact

Loss of rewards for users, locked reward leftovers in contract due to cumulative rounding errors.

Remediation & Fixing Remarks

Use a scaling factor (e.g., 10^{18} or 10^{36}) for reward index computations to preserve precision.

NK-02 Key Findings

Category	Severity	Target	Status
Access Control & Rug-Pull Risk	Low	emergencyWithdraw	Acknowledged

Centralization Risk & Unrestricted Owner Withdrawals

Detailed Description:

The contract contains an owner-restricted administrative function that allows the owner to sweep any token balances from the contract, including user-deposited collateral (**stakingToken**). If the owner key is compromised or malicious, all user funds can be removed immediately without delay.

Code Analysis

```
// FIXING REMARK CODE EXAMPLE
function recoverExcessToken(address tokenAddress, uint256 tokenAmount) external onlyOwner {
    require(tokenAddress != address(stakingToken), "Cannot drain staked assets");
    require(tokenAddress != address(rewardToken), "Cannot drain user reward pool");
    IERC20(tokenAddress).safeTransfer(owner(), tokenAmount);
}
```

Impact Immediate loss of user funds; high protocol trust assumption.

Remediation & Fixing Remarks

Restrict administrative recovery functions to *only* unallocated excess tokens or non-staking/non-reward third-party tokens.

Implement a **Timelock** contract and Multi-Sig wallet for all administrative roles (**OnlyOwner**).

Vulnerability Scan

REENTRANCY

- ✓ No reentrancy risk found
- ✓ Token Interaction Defect

Vulnerability Description

- ✗ The contract uses standard `IERC20.transfer()` and `IERC20.transferFrom()` methods without checking standard boolean return values. Interacting with non-compliant ERC-20 tokens (e.g., USDT, BNB) that do not return a boolean will cause calls to revert or silently fail..
- ✗ **Impact:** Silent transfer failure, leading to unbacked internal balances or stuck transactions.

```
// FIXING REMARK CODE EXAMPLE
using SafeERC20 for IERC20;

// Replaces stakingToken.transferFrom(...)
stakingToken.safeTransferFrom(msg.sender, address(this), amount);
```

Remediation & Fixing Remarks

Import and use OpenZeppelin's `SafeERC20` library with `.safeTransfer()` and `.safeTransferFrom()`

Scanning Line:

Vulnerability Scan

REENTRANCY

- ✓ ERC20.sol (Token Implementation)
- ✓ Missing Deflationary / Fee-on-Transfer Accounting

Vulnerability Description

- ✗ The contract uses standard `IERC20.transfer()` and `IERC20.transferFrom()` methods without checking standard boolean return values. Interacting with non-compliant ERC-20 tokens (e.g., USDT, BNB) that do not return a boolean will cause calls to revert or silently fail..
- ✗ **Impact:** Internal accounting mismatch allowing the last users to fail withdrawals or early stakers to drain funds.

```
// FIXING REMARK CODE EXAMPLE
uint256 balanceBefore = stakingToken.balanceOf(address(this));
stakingToken.safeTransferFrom(msg.sender, address(this), amount);
uint256 balanceAfter = stakingToken.balanceOf(address(this));
uint256 actualDeposited = balanceAfter - balanceBefore;

_balances[msg.sender] += actualDeposited;
_totalSupply += actualDeposited;
```

Remediation & Fixing Remarks

Calculate net deposited amount using balance checks before and after external transfers.

Scanning Line:

Architecture Review

✓ No reentrancy risk found

Vulnerability Description

Vulnerability 2.2: Floating Pragma Versioning

Severity: LOW / INFORMATIONAL

Category: Compiler / Configuration Security

Description:

Contracts use a floating pragma (e.g., `pragma solidity ^0.8.0;`). Contracts deployed with untested or newer compiler versions may introduce unexpected behavior or unaddressed compiler bugs.

Impact: Sub-optimal compiler optimization or exposure to unpatched EVM/Solidity compiler bugs.

Remediation / Fixing Remark:

Lock compiler pragmas to a specific production-tested release version (e.g., `pragma solidity 0.8.20;`).

Vulnerability 2.3: Missing Zero-Address Validation

Severity: LOW / INFORMATIONAL

Category: Input Validation

Description:

Constructor and setters lack checks for `address(0)` when assigning reward distributor, owner, or token contract addresses.

Impact: Accidental deployment bricking or burning of tokens/rewards if set to null address.

Remediation / Fixing Remark:

Add `require(address != address(0), "Zero address validation failed");` checks to constructors and administrative setter functions.

MANUAL REVIEW

NOAH DAO is a community-driven decentralized platform designed to turn the energy of internet culture into an interactive on-chain ecosystem.

Built for the emerging **Arc Network** ecosystem, Noah DAO combines three core experiences: **STAKE. PLAY. FARM. EARN**

TOKEN NAME: NOAH DAO

Ticker: \$NOAH

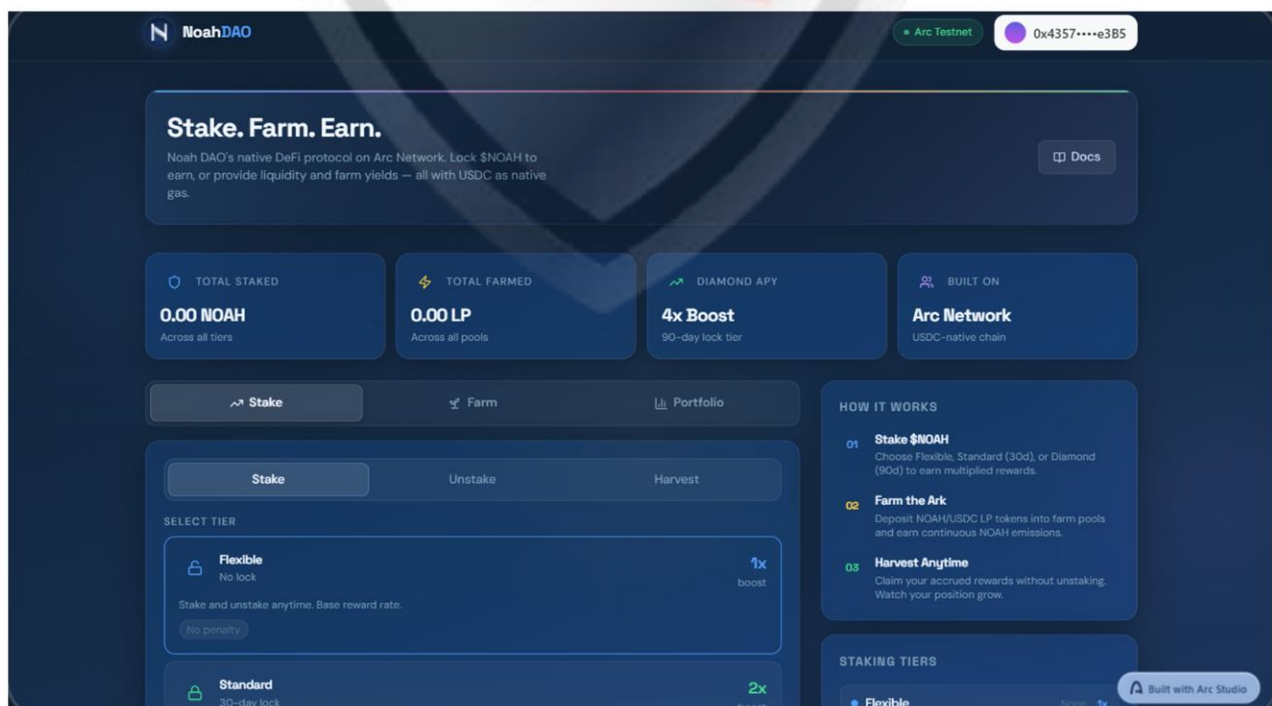
MAX TOTAL SUPPLY: 1,000,000,000,000

Chain/Standard: ARC NETWORK

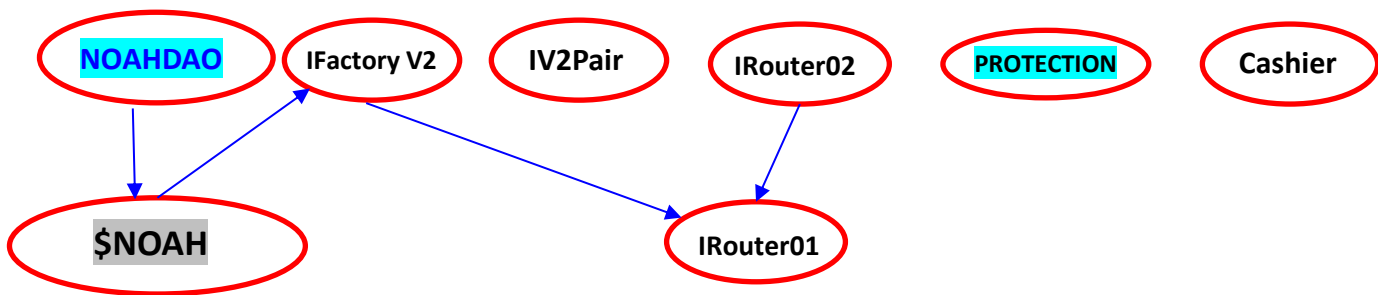
LAUNGUGE: SOLIDITY



The NOAH DAO Platform Is Launched On the ARC Network



INHERITANCE GRAPH



Identifier	Definition	Severity
CEN-12	Centralization privileges of NOAH DAO	Medium #

Vulnerability 0 : No important security issue detected.

Threat level: **Low**

```

function deposit(uint8 poolId, uint256 amount) external nonReentrant {
    if (amount == 0) revert ZeroAmount();
    FarmPool storage pool = _validateAndGetPool(poolId);
    _updateFarmPool(poolId);
    FarmPosition storage position = farmPositions[poolId][msg.sender];
    uint256 pending = _pendingFarmFromPosition(position, pool);
    if (pending > 0) {
        _payoutReward(msg.sender, pending);
        emit FarmHarvested(msg.sender, poolId, pending);
    }
    IERC20(pool.lpToken).safeTransferFrom(msg.sender, address(this), amount);
    position.amount += amount;
    position.rewardDebt = (position.amount * pool.rewardPerShare) / ACC_REWARD_PRECISION;
    pool.totalDeposited += amount;
    totalFarmed += amount;
    emit FarmDeposited(msg.sender, poolId, amount);
}
  
```

ISSUES CHECKING STATUS

Issue Description		Checking Status
1.	Compiler errors.	PASSED
2.	Race Conditions and reentrancy. Cross-Function Race Conditions.	PASSED
3.	Possible Delay In Data Delivery.	PASSED
4.	Oracle calls.	PASSED
5.	Front Running.	PASSED
6.	Sol Dependency.	PASSED
7.	Integer Overflow And Underflow.	PASSED
8.	DoS with Revert.	PASSED
9.	Dos With Block Gas Limit.	PASSED
10.	Methods execution permissions.	PASSED
11.	Economy Model of the contract.	PASSED
12.	The Impact Of Exchange Rate On the solidity Logic.	PASSED
13.	Private use data leaks.	PASSED
14.	Malicious Event log.	PASSED
15.	Scoping and Declarations.	PASSED
16.	Uninitialized storage pointers.	PASSED
17.	Arithmetic accuracy.	PASSED
18.	Design Logic.	PASSED
19.	Cross-Function race Conditions	PASSED
20.	Save Upon solidity contract Implementation and Usage.	PASSED
21.	Fallback Function Security	PASSED



AUDIT RESULT

PASSED

SMART CONTRACT AUDIT OF NOAH DAO

RECOMMENDATION

Deployer and/or contract owner private keys are secured carefully.

Please refer to PAGE-7 CENTRALIZED PRIVILEGES for a detailed understanding.

ALLEVIATION

The [NOAH DAO](#) project team understands the centralization risk. Some functions are provided privileged access to ensure a good runtime behavior in the project



References

- 1 MITRE. CWE-1041: Use of Redundant Code. <https://cwe.mitre.org/data/definitions/1041.html>.
- 2 MITRE. CWE-1099: Inconsistent Naming Conventions for Identifiers. <https://cwe.mitre.org/data/definitions/1099.html>.
- 3 MITRE. CWE-561: Dead Code. <https://cwe.mitre.org/data/definitions/561.html>.
- 4 MITRE. CWE-563: Assignment to Variable without Use. <https://cwe.mitre.org/data/definitions/563.html>.
- 5 MITRE. CWE-663: Use of a Non-reentrant Function in a Concurrent Context. <https://cwe.mitre.org/data/definitions/663.html>.
- 6 MITRE. CWE-837: Improper Enforcement of a Single, Unique Action. <https://cwe.mitre.org/data/definitions/837.html>.
- 7 MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. <https://cwe.mitre.org/data/definitions/841.html>.
- 8 MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- 9 MITRE. CWE CATEGORY: Business Logic Errors. <https://cwe.mitre.org/data/definitions/840.html>.
- 10 MITRE. CWE CATEGORY: Concurrency. <https://cwe.mitre.org/data/definitions/557.html>.
- 11 MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.
- 12 OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor 

Smart contract is interacting with third party protocols e.g., Pancakeswap router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.



RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.

DISCLAIMERS

Vital Block Security provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without InterFi Network's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way

to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED “AS IS” AND “AS AVAILABLE” AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT’S OR ANY OTHER INDIVIDUAL’S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.

LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.



CERTIFICATE BY VITAL BLOCK SECURITY



MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(
        newOwner != address(0),
        "Ownable: new owner is the zero address"
    );
    emit Ownership buned_owner, newOwner);
    _owner = newOwner;
}
```

NOAH DAO

NOAH DAO is a community-driven decentralized platform designed to turn the energy of internet culture into an interactive on-chain ecosystem.

 www.noahdao.xyz

MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER



 **VITALBlock**
security.

MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER

CERTIFICATE

of COMPLIANCE

This certificate is presented to

- This Project Smart Contract Code Has Been Certified By Vital Block Security
- This Safety Certificate Is Only Valid For >>>>>>>>

↖

MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER



MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER



NOAH DAO

SCORE
98

MAXIMUM SCORE ACHIEVED

MAKING LIFE EASIER * EASIER EVERYDAY

 **VITALBlock**
security.



ABOUT VITAL BLOCK

Vital Block provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 1850+ smart contracts, and analyzed Over 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, UI experts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 8+ casual contributors.



1000+
Audits Completed



\$12B+
Secured



1M+
Lines of Code Audited



@vb_audit



@Vitalblock



@vitalblock



github.com/VBS-LABS



Info@vitalblock.org



www.vitalblock.org





Blockchain Security | Smart Contract Audit | KYC Certification | **SAFU**.

MADE IN CANADA